GoodFoods AI Reservation Agent

Goal

Automate the end-to-end reservation process for all GoodFoods locations to reduce staff workload, eliminate manual booking errors, and provide a modern, 24/7 customer service channel.

Long Term Goal

To transform the reservation system from a simple booking utility into an intelligent, revenue-generating platform. This platform will actively reduce revenue loss from no-shows, increase table turnover rates, and build a valuable database of customer preferences to enable personalized marketing and increase customer lifetime value.

Success Criteria & Measurable ROI

This agent's success will be measured by its direct impact on operational efficiency and revenue.

Category	Success Metric (What to Measure)	Potential ROI (The "Why")
Efficiency	Reduction in staff hours spent on phone/email bookings.	Frees up on-site staff to focus on in-person guests, improving service quality and reducing labor costs.
Revenue Loss	Decrease in "no-show" reservation rate.	Recaptures significant lost revenue. A 25% reduction in no-shows for a restaurant doing 100 covers/night could mean thousands in saved revenue per week.
Conversion	Booking completion rate for users who start an interaction.	A 24/7, instant-response agent captures bookings that would be lost overnight or when phone lines are busy.
Data Asset	Number of unique customer profiles created (contact info + booking history).	Creates a new marketing asset. The agent can identify "VIPs" or "at-risk" customers.

Revenue Growth	% increase in off-peak hour bookings (e.g., 4-6 PM weekdays).	The agent's ability to proactively offer deals for quiet times turns empty tables into revenue.

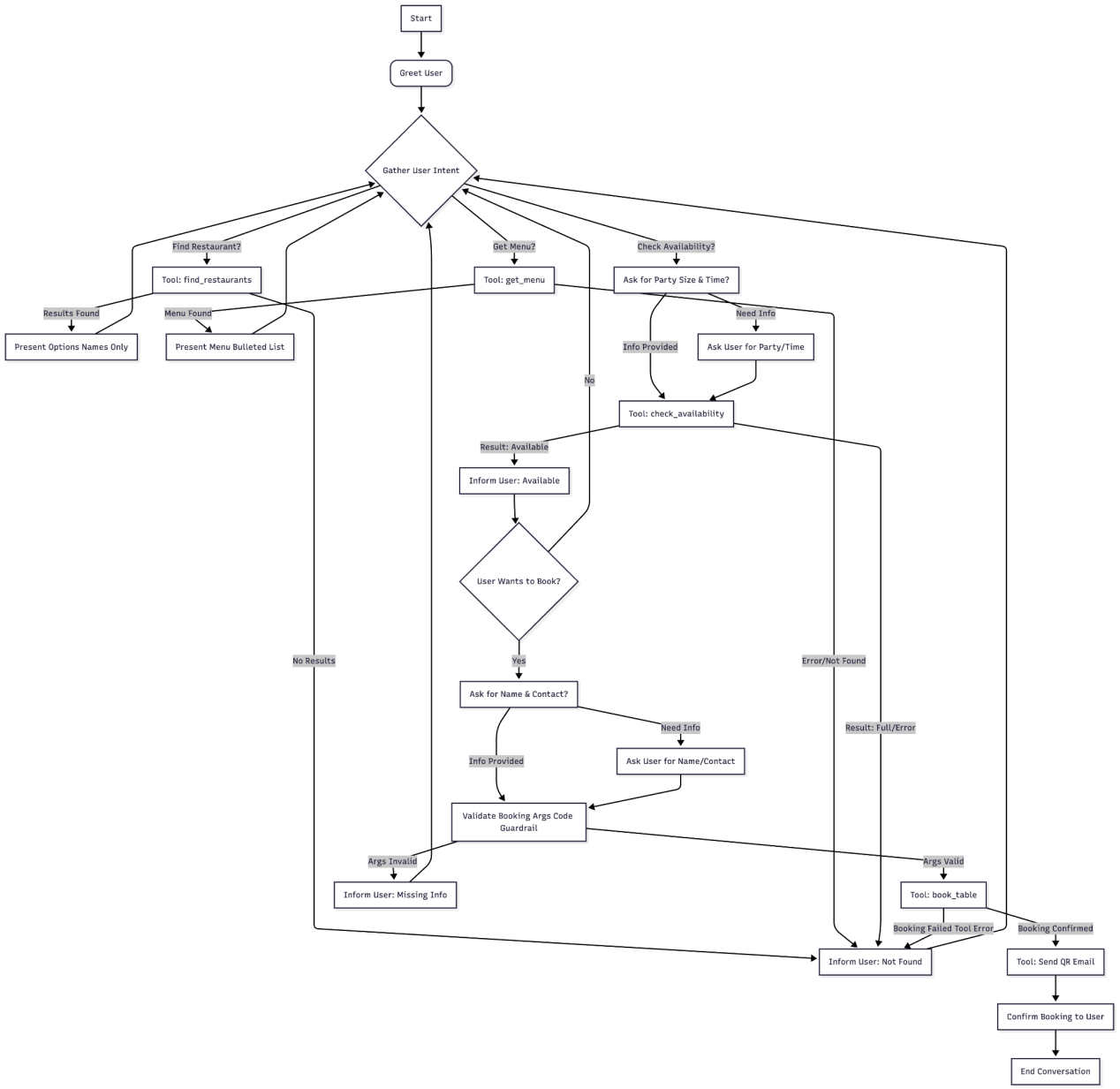
Use Case

A potential customer visits the GoodFoods website. They are greeted by an AI agent and ask, "I'd like a table for 2 at an Indian restaurant tomorrow." The agent, referencing all GoodFoods locations, finds and suggests, "We have 3 Indian locations. GoodFoods - Uptown has an opening at 7 PM. Would you like that?" The user confirms and provides their name and email. The agent finalizes the booking in the system and instantly emails the user a confirmation with a unique QR code, which they can show upon arrival for a fast, frictionless check-in.

Key Steps (End-User Agent Flow)

1. **Greet & Discover:** The user starts a chat. The agent greets them and asks how it can help.
2. **Recommend & Refine:** The user states their need (e.g., "a cheap Italian place"). The agent uses its tools (find_restaurants) to search its database by cuisine, location, and even budget (by checking menus). It presents 3-4 smart options.
3. **Explore & Check:** The user asks to see the menu for an option. The agent uses get_menu (which presents the menu sorted by price). The user then asks, "Is a table for 4 at 8 PM available?" The agent uses check_availability to give a real-time "yes" or "no."
4. **Gather & Confirm:** The user decides to book. The agent asks for the required details it's missing (e.g., "Great! And what's your name and email address?").
5. **Finalize & Deliver:** The agent calls the book_table tool. Upon success, it confirms the booking with the user and immediately triggers the _send_confirmation_email function, sending the QR code and booking details to the customer.

State Transition Diagram



Agent Features

Key specifications that the customer cares about

- **24/7 Automated Bookings:** The agent can accept and confirm reservations instantly, any time of day, without human intervention.
- **Intelligent Restaurant Discovery:** Customers can find restaurants by asking for vague preferences like "a cheap Italian place" or "an Indian restaurant downtown."

- **Budget-Aware Recommendations:** The agent can answer "What's on the menu under Rs.200?" by using the `get_menu` tool (which sorts by price) and filtering the results.
- **Real-Time Availability Checks:** The agent provides an instant "yes" or "no" on table availability for a specific party size and time.
- **Automated QR Code Confirmation:** Customers receive an immediate email confirmation with a unique QR code for a frictionless, high-tech check-in.
- **Conversational Recovery:** The agent is robust. If a user asks to book without providing all details, the agent intelligently asks for the *specific* missing information (e.g., "Sure, for how many people and at what time?") instead of failing.

KBs (Knowledge Bases)

- **Restaurant & Menu Database:** The agent's core knowledge is sourced from a single "database" (currently `restaurants.json`). This KB contains all 50-100 restaurant locations, their addresses, specific cuisines, seating capacities, and full itemized menus with prices.

Does it need tools?

- **Yes, critically.** The agent is built on a "tool-calling architecture." The Cerebras LLM acts as the "brain" to decide *which* Python function (tool) to call in response to a user's request.
- **Key Tools Used:**
  - **`find_restaurants`**: Searches the database.
  - **`get_menu`**: Retrieves a specific menu (sorted by price).
  - **`check_availability`**: Checks table inventory.
  - **`book_table`**: Finalizes a booking and writes it to the database.
  - **`_send_confirmation_email`**: A helper function called by `book_table` to send the QR code email.

Languages & Frameworks

- Python & LangChain

Integrations

- **Cerebras API:** For all LLM-based decisions and conversation.
- **GoodFoods Database:** The central **`restaurants.json`** file (or a future SQL database) that holds all locations, menus, and bookings.

- **Email SMTP Server:** To send the confirmation emails.

New Features:

Feature	Specification	Difficulty
Core Knowledge	Bot is "aware" of all 50-100 GoodFoods locations, including their cuisine, capacity, and menus.	**Green**
Tool-Enabled	Bot *must* use tools to interact with the database. It cannot "hallucinate" bookings.	**Green**
Smart Recommendations	Bot can recommend restaurants based on cuisine, location, and **budget** (by cross-referencing `get_menu`).	**Green**
Automated QR Code Confirmation	Upon successful booking, bot *must* email a confirmation with a unique QR code to the user.	**Green**
Conversational Repair	Bot *must* be able to ask for missing information (e.g., "For how many people?") without failing.	**Green**
Waitlist Management	If a table is full, the bot offers to add the user to a waitlist and auto-notifies them if a slot opens.	**Yellow**
Proactive Upselling	**(Opportunity)** If a slot is full, offer a deal on an off-peak time (e.g., "7 PM is full, but I can offer a 20% discount on appetizers if you book for 6 PM").	**Yellow**

Full CRM Integration	**(Opportunity)** Bot links a user's contact info to their booking history, identifying them as a "VIP" on their 5th visit and offering a proactive reward.	**Red**

Key Challenges

1. **LLM "Leaking" / Hallucination:** As seen during development, the LLM will constantly try to break JSON format or make up answers. This requires a proper parsing in `agent.py` and extremely strict `SYSTEM_PROMPT` rules.
2. **Unstructured User Input:** Customers will be unpredictable. They will change their minds ("Wait, not 4 people, 5"), ask unrelated questions ("What's the parking like?"), or provide all details at once. The agent must be robust enough to handle this.
3. **Email Deliverability:** Ensuring QR code emails land in the user's inbox and not their spam folder is a significant technical challenge that requires proper SMTP setup and domain authentication (DKIM, SPF).
4. **Real Time Inventory Sync:** The current reliance on `restaurants.json`, or even a standard database, introduces a critical risk of concurrency conflicts. If a walk-in customer takes a table, and that event is not updated in the database *before* the AI agent queries it, a data-desynchronization occurs. This can cause potential problems, such as the agent booking a table that is no longer available, leading to a severe customer service failure.